

Московский Авиационный Институт
(Национальный Исследовательский Университет)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра №806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу
«Операционные системы»

Группа:	М8О-208БВ-25
Студент:	Андрушко К.М.
Преподаватель:	Бахарев В.Д.
Оценка:	_____
Дата:	__.__.26

Москва, 2026

Постановка задачи

Вариант 1 (группа вариантов 1).

Цель работы. Приобретение практических навыков в управлении процессами в ОС и обеспечении обмена данными между процессами посредством каналов.

Задание. Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должна создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Условие варианта. Родительский процесс создаёт дочерний процесс. Первой строкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передаёт команды пользователя через pipe1, который связан со стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передаёт данные в родительский процесс через pipe2. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Пользователь вводит команды вида «число число число<endline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип int. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

`pid_t fork(void)` – создаёт дочерний процесс, копию вызывающего. В родителе возвращает PID ребёнка, в ребёнке 0, при ошибке -1.

`int execve(const char *filename, char *const argv[], char *const envp[])` – заменяет образ памяти процесса программой из указанного файла.

Используется, чтобы дочерний процесс стал отдельной программой `child` и получил имя файла в `argv[1]`.

`pid_t waitpid(pid_t pid, int *status, int options)` – ожидает завершения дочернего процесса и возвращает информацию о том, как он завершился.

`int pipe(int pipefd[2])` – создаёт неименованный канал: `pipefd[0]` конец для чтения, `pipefd[1]` конец для записи.

`int dup2(int oldfd, int newfd)` – делает так, чтобы `newfd` указывал на то же, что и `oldfd`. Используется для подключения канала к стандартному входу дочернего процесса.

`int open(const char *pathname, int flags, mode_t mode)` – открывает или создаёт файл. Дочерний процесс создаёт выходной файл с флагами `O_WRONLY | O_CREAT | O_TRUNC`.

`int close(int fd)` – закрывает файловый дескриптор.

`ssize_t read(int fd, void *buf, size_t count)` – читает данные из дескриптора (клавиатура, канал).

`ssize_t write(int fd, const void *buf, size_t count)` – записывает данные в дескриптор (канал, файл, поток ошибок).

`void _exit(int status) / void exit(int status)` – завершают процесс и возвращают код статуса.

`signal(SIGPIPE, SIG_IGN)` – отключает аварийное завершение процесса по сигналу `SIGPIPE` при записи в канал без читателя: вместо этого `write` возвращает ошибку, которую можно обработать.

Решение состоит из двух отдельных программ, которые компилируются независимо: родительской (`parent.c`) и дочерней (`child.c`). Использовались только системные вызовы, функции библиотеки `stdio.h` не применялись: ввод и вывод выполняются через `read` и `write`, а числа переводятся в текст и обратно вручную.

Алгоритм работы родительского процесса:

1. Прочитать первую строку ввода (имя файла) по одному байту, чтобы не захватить следующие строки, которые предназначены дочернему процессу.
2. Создать канал `pipe1` до вызова `fork`, чтобы оба процесса получили его концы.
3. Вызвать `fork`. В дочернем процессе закрыть конец `pipe1` для записи, вызовом `dup2` подключить конец для чтения к стандартному входу (дескриптор 0), закрыть оставшийся дескриптор и запустить `./child` через `execve`, передав имя файла в `argv[1]`.
4. В родительском процессе закрыть конец `pipe1` для чтения и в цикле читать ввод пользователя, пересылая его без изменений в `pipe1`.
5. При окончании ввода (`Ctrl+D`) закрыть конец `pipe1` для записи, чтобы дочерний процесс получил признак конца данных, затем вызвать `waitpid` и проверить код завершения дочернего процесса.

Алгоритм работы дочернего процесса:

1. Проверить, что имя файла передано в `argv[1]`, и открыть (создать) файл вызовом `open`.
2. В цикле читать данные со стандартного входа (это `pipe1`) и разбирать их посимвольно: цифры накапливаются в текущее число, знак минус запоминается, пробел завершает число и прибавляет его к сумме строки.
3. По символу конца строки записать сумму строки в файл (число переводится в текст вручную) и обнулить накопленные значения. Строки с некорректными символами пропускаются с сообщением об ошибке, пустые строки игнорируются.
4. Сумма хранится в 64-разрядном типе, чтобы сумма произвольного количества значений типа `int` не переполнялась.
5. При получении признака конца данных (`read` вернул 0) обработать последнюю строку, если она пришла без символа конца строки, закрыть файл и завершиться.

Канал `pipe2` в данном варианте не использовался: дочернему процессу нечего передавать родительскому, так как результат записывается прямо в файл.

Обработка ошибок: проверяются возвращаемые значения `pipe`, `fork`, `dup2`, `execve`, `open`, `close`, `read`, `write` и `waitpid`. При ошибке выводится сообщение на русском языке в поток ошибок и процесс завершается с ненулевым кодом. Родительский процесс проверяет код завершения дочернего.

Код программы

parent.c

```
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

#define DIE(msg) do { \
    write(STDERR_FILENO, msg, sizeof(msg) - 1); \
    exit(EXIT_FAILURE); \
} while (0)

int main(void) {
    char name[256];
    int len = 0;
    char c;

    while (len < (int)sizeof(name) - 1 && read(STDIN_FILENO, &c, 1) == 1 && c != '\n') {
        name[len++] = c;
    }
    name[len] = '\0';
    if (len == 0) {
        DIE("ошибка: пустое имя файла\n");
    }

    signal(SIGPIPE, SIG_IGN);

    int pipe1[2];
    if (pipe(pipe1) == -1) {
        DIE("ошибка: не удалось создать канал\n");
    }

    pid_t pid = fork();
    if (pid == -1) {
        DIE("ошибка: не удалось создать процесс (fork)\n");
    }

    if (pid == 0) {
        close(pipe1[1]);
        if (dup2(pipe1[0], STDIN_FILENO) == -1) {
            DIE("ошибка: не удалось выполнить dup2\n");
        }
        close(pipe1[0]);

        char *args[] = {"child", name, NULL};
        char *env[] = {NULL};
        execve("./child", args, env);

        const char msg[] = "ошибка: не удалось запустить дочернюю программу (execve)\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        _exit(EXIT_FAILURE);
    }

    close(pipe1[0]);

    char buf[4096];
    ssize_t n;
    int failed = 0;

    while ((n = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        if (write(pipe1[1], buf, n) != n) {
            failed = 1;
            break;
        }
    }
    if (n < 0) {
        failed = 1;
    }
}
```

```

    }

    close(pipe1[1]);

    int status;
    if (waitpid(pid, &status, 0) == -1) {
        DIE("ошибка: не удалось дождаться дочернего процесса\n");
    }

    if (failed) {
        DIE("ошибка: не удалось передать данные дочернему процессу\n");
    }
    if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
        DIE("ошибка: дочерний процесс завершился с ошибкой\n");
    }

    return 0;
}

```

child.c

```

#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>
#include <stdint.h>

#define DIE(msg) do { \
    write(STDERR_FILENO, msg, sizeof(msg) - 1); \
    exit(EXIT_FAILURE); \
} while (0)

static int file;
static int64_t sum, cur;
static int in_num, neg, has_nums, bad;

static void write_sum(int64_t v) {
    char out[32];
    int pos = sizeof(out);
    uint64_t u = v < 0 ? -(uint64_t)v : (uint64_t)v;

    out[--pos] = '\n';
    do {
        out[--pos] = '0' + u % 10;
        u /= 10;
    } while (u > 0);
    if (v < 0) {
        out[--pos] = '-';
    }

    ssize_t size = sizeof(out) - pos;
    if (write(file, out + pos, size) != size) {
        DIE("ошибка: не удалось записать в файл\n");
    }
}

static void end_number(void) {
    if (in_num) {
        sum += neg ? -cur : cur;
        has_nums = 1;
    } else if (neg) {
        bad = 1;
    }
    cur = 0;
    in_num = 0;
    neg = 0;
}

static void end_line(void) {
    end_number();
    if (bad) {

```

```

        const char msg[] = "ошибка: некорректная строка пропущена\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
    } else if (has_nums) {
        write_sum(sum);
    }
    sum = 0;
    has_nums = 0;
    bad = 0;
}

int main(int argc, char **argv) {
    if (argc < 2) {
        DIE("ошибка: не указано имя файла\n");
    }

    file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file == -1) {
        DIE("ошибка: не удалось открыть файл\n");
    }

    char buf[4096];
    ssize_t n;

    while ((n = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        for (ssize_t i = 0; i < n; ++i) {
            char c = buf[i];

            if (c >= '0' && c <= '9') {
                in_num = 1;
                cur = cur * 10 + (c - '0');
            } else if (c == '-' && !in_num && !neg) {
                neg = 1;
            } else if (c == ' ' || c == '\t' || c == '\r') {
                end_number();
            } else if (c == '\n') {
                end_line();
            } else {
                bad = 1;
            }
        }
    }

    if (n < 0) {
        DIE("ошибка: не удалось прочитать данные из stdin\n");
    }

    end_line();

    if (close(file) == -1) {
        DIE("ошибка: не удалось закрыть файл\n");
    }
    return 0;
}

```

Протокол работы программы

Сборка программ:

```
$ gcc parent.c -o parent
$ gcc child.c -o child
```

Тестирование (^D означает нажатие Ctrl+D):

Тест 1. Обычный ввод, несколько строк, отрицательные числа

```
$ ./parent
result.txt
1 2 3
10 -4 5
-2147483648 -2147483648 7
100
^D
$ cat result.txt
6
11
-4294967289
100
```

Тест 2. Строка с некорректным символом пропускается

```
$ ./parent
r2.txt
4 x 5
ошибка: некорректная строка пропущена
7 8
^D
$ cat r2.txt
15
```

Тест 3. Нули, пустая строка, последняя строка без Enter

```
$ ./parent
r3.txt
0

-5 5
-0 0
42^D
$ cat r3.txt
0
0
0
42
```

Тест 4. Пустое имя файла

```
$ ./parent

ошибка: пустое имя файла
$ echo $?
1
```

Тест 5. Отсутствует исполняемый файл ./child

```
$ ./parent
a.txt
1 2
ошибка: не удалось запустить дочернюю программу (execve)
ошибка: дочерний процесс завершился с ошибкой
$ echo $?
1
```

Тест 6. Недоступный путь к файлу

```
$ ./parent  
/nonexistent/x.txt  
1 2  
ошибка: не удалось открыть файл  
ошибка: дочерний процесс завершился с ошибкой  
$ echo $?  
1
```

Вывод

В ходе лабораторной работы изучены системные вызовы `fork`, `execve`, `waitpid`, `pipe` и `dup2`, реализованы две отдельные программы, обменивающиеся данными через неименованный канал. Родительский процесс передаёт введённые строки дочернему, а тот считает сумму чисел и записывает её в файл. Основные трудности вызвали закрытие неиспользуемых концов канала и разбор чисел без `stdio.h`. Программа обрабатывает системные ошибки и некорректный ввод.